

SRE Lab

A hands-on DevOps and SRE troubleshooting lab with five full-stack applications on Amazon EKS. The applications use Amazon RDS for PostgreSQL, public traffic enters through Route 53 and a shared Application Load Balancer, images live in ECR, and Datadog provides metrics, events, logs, traces, dashboards, and monitors.

The training loop is:

```
Datadog symptom -> scope -> Kubernetes or AWS evidence -> root cause -> fix -> Datadog recovery
```

Launch the lab

Complete these steps in order. The deployment creates billable AWS resources, so review the prerequisites and keep the teardown command available before you begin.

1. Create the Datadog keys before launching

The lab requires both keys at launch:

- `DATADOG_API_KEY` lets the Datadog Agent send metrics, events, logs, and traces.
- `DATADOG_APP_KEY` lets the setup script import the included dashboards and monitors and provision Real User Monitoring (RUM).

In Datadog, create or obtain an API key and an application key for your account. Store them securely and never add them to `terraform.tfvars`, a source file, or Git. Confirm the correct Datadog site for your account, such as `datadoghq.com` or `datadoghq.eu`.

2. Check the prerequisites

- Bash 4.4 or newer
- Terraform, AWS CLI, kubectl, Docker, Helm, jq, `envsubst`, `curl`, and `nslookup`
- An AWS account with credentials configured
- An existing Route 53 public hosted zone with registrar delegation already configured
- A Datadog account, API key, and application key
- Budget for real EKS, EC2, NAT Gateway, ALB, Route 53, and RDS resources

The fixed resource names support one lab deployment per AWS account. Run the commands from Bash on Linux, WSL, or macOS. macOS users need a newer Bash than the system Bash 3.2.

Verify the most important connections before launching:

```
aws sts get-caller-identity
docker version
terraform version
kubect1 version --client
helm version
```

3. Configure the domain

```
cp terraform/terraform.tfvars.example terraform/terraform.tfvars
```

Open `terraform/terraform.tfvars` and set `dns_zone_name` to the Route 53 public hosted zone that will host the five application records.

4. Launch the complete environment

Replace the placeholder values and run the command from the repository root. Do not save real keys in the command history of a shared computer.

```
DATADOG_API_KEY=<api-key> \
DATADOG_APP_KEY=<app-key> \
DATADOG_SITE=datadoghq.com \
./scripts/setup.sh
```

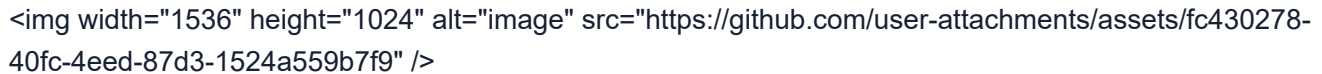
`setup.sh` validates prerequisites, provisions the Terraform resources, builds and pushes ten images, initializes the application databases, deploys the Kubernetes workloads, configures ALB and Route 53, installs metrics-server, and installs Datadog. When both Datadog keys are supplied, it also imports the included dashboards and monitors and provisions RUM. The script prints each application URL when it finishes.

5. Verify the deployment

```
aws eks update-kubeconfig --name sre-lab --region us-east-1
kubect1 get nodes
kubect1 get deployments -A
kubect1 get pods -A
```

All application Deployments should have their desired Ready replicas. Open the five URLs printed by `setup.sh`, then confirm that data appears in the Datadog application dashboards and **SRE Lab Scenario Signals** before triggering an incident.

Architecture



Five applications share the EKS cluster: `ecommerce`, `banking`, `food-delivery`, `student-portal`, and `support-tickets`. Each has a React/nginx frontend and Node.js/Express backend. See [Architecture](#) for the full design and training tradeoffs.

Repository layout

<code>apps/</code>	Application source and Kubernetes manifests
<code>datadog/</code>	Helm values, dashboards, and monitors
<code>docs/</code>	Student, instructor, scenario, runbook, and PDF material
<code>ingress/</code>	One hostname-based Ingress per application
<code>namespaces/</code>	Namespaces, quotas, and limit ranges
<code>scripts/</code>	Setup, teardown, PDF generation, and chaos triggers
<code>terraform/</code>	VPC, EKS, ECR, RDS, Route 53, IAM, and ALB controller

Access applications

The configured domain is saved in `.lab-domain`.

```
open https://ecommerce.${cat .lab-domain}
open https://banking.${cat .lab-domain}
open https://food-delivery.${cat .lab-domain}
open https://student-portal.${cat .lab-domain}
open https://support-tickets.${cat .lab-domain}
```

Datadog

`datadog/helm-values.yaml` enables APM, container logs, Kubernetes events, infrastructure metrics, and the Cluster Agent. Setup imports every JSON file under `datadog/dashboards/` and `datadog/monitors/` when both Datadog keys are supplied.

Establish a healthy baseline in each application dashboard and **SRE Lab Scenario Signals** before starting an exercise.

Five incident scenarios

Scenario	Main DevOps skill
Bad Container Image Deployment	ECR and Kubernetes Deployments
Bad ConfigMap Rollout	ConfigMaps and health probes
OOMKilled and Resource Limits	Kubernetes resource management
ALB and Ingress Routing Failure	ALB and Kubernetes networking
High Application Latency	Datadog APM and workload troubleshooting

Student scenarios begin with symptoms and Datadog. Solutions and instructor triggers are in [Instructor Answer Keys](#). Production-style STAR versions are in [DevOps STAR Scenarios](#).

Guides and PDFs

- [Student Guide](#)
- [Instructor Answer Keys](#)
- [Scenario test checklist](#)
- [Runbooks](#)
- [SLO, SLA, and SLI](#)
- [Error budgets](#)
- [SRE Lab launch guide PDF](#)
- [DevOps STAR scenario PDF](#)
- [Combined scenario PDF](#)
- [Individual scenario PDFs](#)

The README, DevOps STAR document, and Markdown files under `docs/incident-scenarios/` are the PDF sources. Regenerate PDFs with:

```
powershell -ExecutionPolicy Bypass -File scripts/generate-pdfs.ps1
```

Destroy the environment

The lab costs money while running. Teardown deletes Ingress resources first so the controller can release the ALB, uninstalls Datadog and the controller, removes namespaces, and runs Terraform destroy.

```
./scripts/teardown.sh
```

Afterward, verify that the lab has no remaining ALBs, target groups, NAT Gateways, Elastic IPs, ECR repositories, or RDS instances in the AWS console.